

Contents

1 day 6: apis	1
1.1 dict data type	1
1.2 the MET API	2
1.2.1 1. the anatomy of an API request:	2
1.2.2 2. make the request using requests.get()	2
1.2.3 3. inspecting objects:	2
1.2.4 4. making a new call, using the “objects” endpoint . .	3
1.3 the order of tasks:	4
1.4 BREAK	4
1.5 collecting several objects	4
1.6 google the error for a solution	6
1.7 saving to DataFrame	6
1.8 about “women”	6
1.9 reading the docs: partners challenge	6
1.10 start challenge, to make requests using multiple parameters .	7

1 day 6: apis

Agenda

- begin work with APIs
 - dictionary structures - new data structure
 - the MET API docs - beginner friendly
 - end goal: to get data from MET databases into our own spreadsheet, curate that data a little bit.

1.1 dict data type

What is a dict? key:value pairs

```
filipa = {  
    'name': ['filipa', 'maria', 'da gama', 'barreiros', 'calado'],  
    'sign': ['gemini', 'saggitarius', 'aquarius'],  
    'interests': ['literature', 'programming', 'soccer']  
}
```

access items with brackets, the 'key'

```

    filipa['interests']

    # access items within a column, like list indexing

    filipa['interests'][2]
}

```

1.2 the MET API

1.2.1 1. the anatomy of an API request:

3 parts of an API: the base url, the path, the query.

```

base_url = "https://collectionapi.metmuseum.org" # root of the website
path = "/public/collection/v1.1/search" # directory structure, folders
query = "?q=woman" # parameter, endpoint - keyword/specifications

```

1.2.2 2. make the request using requests.get()

```

# our request
women = requests.get(f'{base_url}{path}{query}')

# should say 200
women

```

All you need for an API is a URL. An HTTP request.

Some URLs contain requests embedded inside of them. If you know how to look.

1.2.3 3. inspecting objects:

Inspecting: start with two functions: type() and dir()

Using type() and dir() to better understand our response data. The end goal is to sift through the data to discover interesting things.

```

# what type of object do we have?

type(women)

```

```
# What can we do with a Response object? Spend a couple of minutes exploring the differ
dir(women)
```

Then, use `.json()` to parse our response object, and `.keys()` to see the keys (labels).

```
# why do we need to add parenthesis?
parsed = women.json()

# what is it
type(parsed)

# we can use .keys() to get the keys

parsed.keys()
```

How do we get data from a dictionary?

```
# formatting to get just the objectID key

parsed['objectIDs']

# what are we looking at?

# how would we get just the first object id?

first = parsed['objectIDs'][0]
```

1.2.4 4. making a new call, using the “objects” endpoint

We are going to use another endpoint to get further information, using the data we’ve already requested.

So far, we’ve been using the "search" path. Now, we will use the "objects" parth.

We can use this first object to create a new request.

```
objects_path = '/public/collection/v1.1/objects/'

url = f'{base_url}{objects_path}{first}'

first_object = requests.get(url)
```

Inspecting our new response object, parsing the results

```
# checking the resulting object
first_object

# the response is just like before.
type(first_object)
dir(first_object)

# parsing the object:

first_object.json()
```

See all the data about this object.
See the objectURL? Copy and paste it into your browser.

1.3 the order of tasks:

1. construct the API request (URL)
2. make the request, `requests.get()`
3. inspect and parse our response using `type()`, `.json()`, `.keys()`
4. make another call to get more information.

1.4 BREAK

1.5 collecting several objects

What if we want to collect data on several objects, and save them to a spreadsheet, with columns like "artist", "title", etc.?

- write a loop that makes calls to each objectID, parses the response, and adds to a list. (Will end with a list of dicts)

Write it out in pseudocode

- make a list of objectIDs
- write a loop
- make call to objectID
- parse response
- append response to list

```
from time import sleep

# just the first ten
ids = parsed['objectIDs'][:10]

# start small, just print the ids for now
for item in ids:
    print(item)

# add pseudocode, then fill it in
# what are the steps we need to take?

first_ten = []
for item in ids:

    # passing the objectID variable into the URL
    base_url = 'https://collectionapi.metmuseum.org'
    path = '/public/collection/v1/objects/'
    url = f'{base_url}{path}{item}'

    # grabbing our response for that object
    response = requests.get(url)

    # parsing our response with json
    parsed = response.json()

    sleep(1)
```

```

    # appending the response to our new list
    first_ten.append(parsed)

# check the content.
type(first_ten)

```

Now, we have a list object with tons of information that we can extract from.

How can we extract this information? Let's say, the artist name?

```

for item in first_ten:
    print(item['artistDisplayName'])

```

This gives us an error

1.6 google the error for a solution

There are multiple ways

```

for item in first_ten:
    if item.get('artistDisplayName'):
        print(item['artistDisplayName'])

```

1.7 saving to DataFrame

- save to lists
- save to df
- save to csv

1.8 about “women”

Try it with gender. Why do you think we see only female?

1.9 reading the docs: partners challenge

Look at the docs, see the different paths and parameters.

Implement the search path with an additional parameter for the search endpoint. And then, take the results of that search and look for the objects.

1.10 start challenge, to make requests using multiple parameters